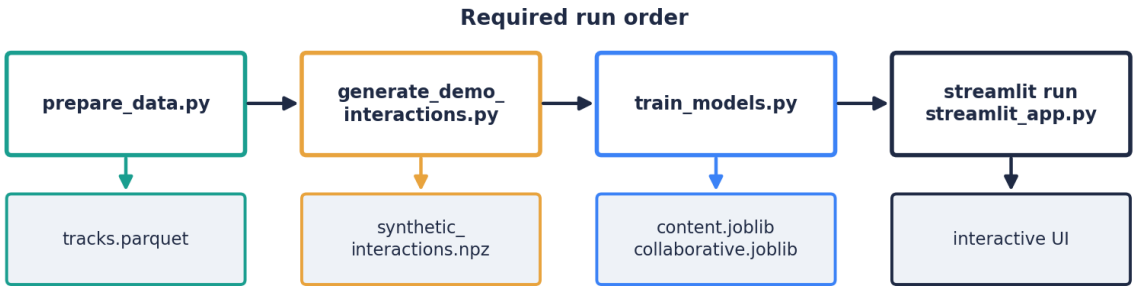
PROJECT REPORT

Hybrid Music Recommendation System

Combining content-based similarity with item-item collaborative filtering to recommend the top 10 tracks from a single song or genre input



Prepared by	Nirmit Gupta
Date	23 October 2026
Domain	Recommender systems / Machine learning
Technology	Python, pandas, scikit-learn, SciPy, Streamlit, Joblib, PyArrow
Data	Spotify track metadata (Kaggle) + reproducible synthetic interactions

Abstract

This report documents the design, implementation and evaluation approach of a hybrid music recommendation system. The system accepts exactly one input, either a selected song or a selected genre, and returns ten recommended tracks through a Streamlit web interface. Recommendations are produced by two complementary engines. A **content-based engine** measures similarity between tracks using standardised audio features, one-hot encoded genre and TF-IDF artist features with cosine nearest-neighbour search. An **item-item collaborative filtering engine** scores tracks using cosine similarity over a sparse user-track interaction matrix. The two score sets are min-max normalised within the candidate set and blended with a tunable weight *alpha*.

The source catalog is a Spotify track-metadata dataset from Kaggle that contains no real listening histories. The project therefore generates reproducible, genre-biased **synthetic** interactions to demonstrate and test the collaborative component, and it states this limitation openly in the code, the user interface and the documentation. The report covers the data and preprocessing pipeline, system architecture, algorithms, module-level implementation, ranking metrics, alpha-tuning procedure, limitations and future work.

Contents

1. Introduction **4**
1.1 Problem statement . . . 4
1.2 Objectives . . . 4
1.3 Scope . . . 4
1.4 Key features at a glance . . . 4
2. Background and Related Concepts **5**
2.1 Content-based filtering . . . 5
2.2 Item-item collaborative filtering . . . 5
2.3 Hybrid recommenders . . . 5
2.4 Ranking metrics . . . 5
3. Dataset and Preprocessing **5**
3.1 Source data . . . 5
3.2 Preprocessing pipeline . . . 6
3.3 Content feature set . . . 7
4. System Architecture **8**
4.1 Technology stack . . . 8
4.2 Project structure . . . 9
5. Methodology **9**
5.1 Content-based engine . . . 9
5.2 Synthetic interaction generation . . . 10
5.3 Item-item collaborative filtering engine . . . 11
5.4 Hybrid ranking . . . 11
5.5 Recommend by Song . . . 12
5.6 Recommend by Genre . . . 13
6. Implementation **14**
6.1 Module responsibilities . . . 14
6.2 Execution workflow . . . 14
6.3 Training and persistence . . . 14
6.4 Streamlit application . . . 14
6.5 Hyper-parameters and defaults . . . 15
7. Evaluation Methodology **15**
7.1 Metrics . . . 15
7.2 Held-out protocol . . . 16
7.3 Alpha tuning . . . 16
7.4 Reported results . . . 17
8. Discussion **17**
8.1 Design strengths . . . 17
8.2 Observations on the current implementation . . . 17
9. Limitations **18**
10. Future Work **18**
11. Conclusion **18**
References **19**
Appendix A. Setup and Run Commands **20**
Appendix B. Source Code Listings (Screenshots) **20**

1. Introduction

1.1 Problem statement

Music catalogs now contain tens of millions of tracks, so listeners cannot browse them exhaustively. Recommender systems help by ranking a small set of candidates that match a stated or inferred taste. Two classical families exist. Content-based methods recommend items whose attributes resemble a seed item. Collaborative methods recommend items that tend to be consumed by the same users. Each has a characteristic weakness: content-based filtering cannot capture taste signals that are absent from the metadata, while collaborative filtering needs interaction data and struggles with items that have few interactions. A hybrid approach aims to combine their strengths.

1.2 Objectives

- Build a recommender that returns the **top 10 tracks** from **one input**: a selected song or a selected genre.
- Implement a **content-based engine** over audio features, genre and artist.
- Implement a scalable **sparse item-item collaborative filtering** engine.
- Combine both engines through a **normalised, weighted hybrid score** with a tunable parameter alpha.
- Provide an interactive **Streamlit** interface that exposes the content, collaborative and hybrid scores for transparency.
- Provide an **evaluation and tuning** workflow based on ranking metrics (Precision, Recall, MAP, NDCG, coverage).
- Be honest about data limitations: collaborative signals are **synthetic** and are not real Spotify listener behaviour.

1.3 Scope

The project covers the full pipeline from raw CSV to an interactive application. It does not use any real user histories, does not personalise per user, and does not claim to estimate real-world recommendation quality. Its purpose is to demonstrate a complete, reproducible, well-structured hybrid recommender architecture.

1.4 Key features at a glance

Feature	Description
Two input modes	Recommend by Song (seed track excluded from results) or by Genre (results stay inside the genre).
Hybrid ranking	alpha x normalised content score + (1 - alpha) x normalised collaborative score.
Transparent output	The UI shows Song, Artist, Genre, Popularity, Content, Collaborative and Hybrid scores.
Sparse computation	No dense catalog-wide similarity matrix is built; scores are computed per seed from sparse matrices.
Reproducibility	Synthetic interactions use a fixed seed (42); evaluation and tuning use held-out events.
Honest evaluation	Metrics are computed on held-out synthetic events and described as pipeline demonstrations only.

2. Background and Related Concepts

2.1 Content-based filtering

Content-based filtering represents each item as a feature vector and recommends items near the seed in that space. Here, numeric audio descriptors (for example danceability, energy and valence) are standardised, genre is one-hot encoded, and artist names are converted to TF-IDF vectors (Salton and Buckley, 1988). Similarity is measured with cosine similarity, which depends on the angle between vectors rather than their magnitude.

2.2 Item-item collaborative filtering

Item-based collaborative filtering (Sarwar et al., 2001) computes similarities between item columns of a user-item matrix. Two tracks are similar when many of the same users interacted with both. For a binary matrix M with users as rows and tracks as columns, the cosine similarity between items i and j is the dot product of their columns divided by the product of their Euclidean norms. Item-item methods are attractive because item neighbourhoods are comparatively stable and can be computed from sparse data.

2.3 Hybrid recommenders

Hybrid recommenders (Burke, 2002) combine multiple techniques to offset individual weaknesses. A *weighted* hybrid, used in this project, blends the scores of component recommenders with fixed or tuned weights. Because component scores live on different scales, they must be normalised before blending; this project applies min-max normalisation within the candidate set.

2.4 Ranking metrics

Top-N recommenders are evaluated with ranking metrics rather than rating-error metrics. Precision@K and Recall@K measure hit counts in the top K; MAP@K rewards placing relevant items early; NDCG@K (Jarvelin and Kekalainen, 2002) discounts hits logarithmically by rank; catalogue coverage and diversity describe breadth rather than accuracy.

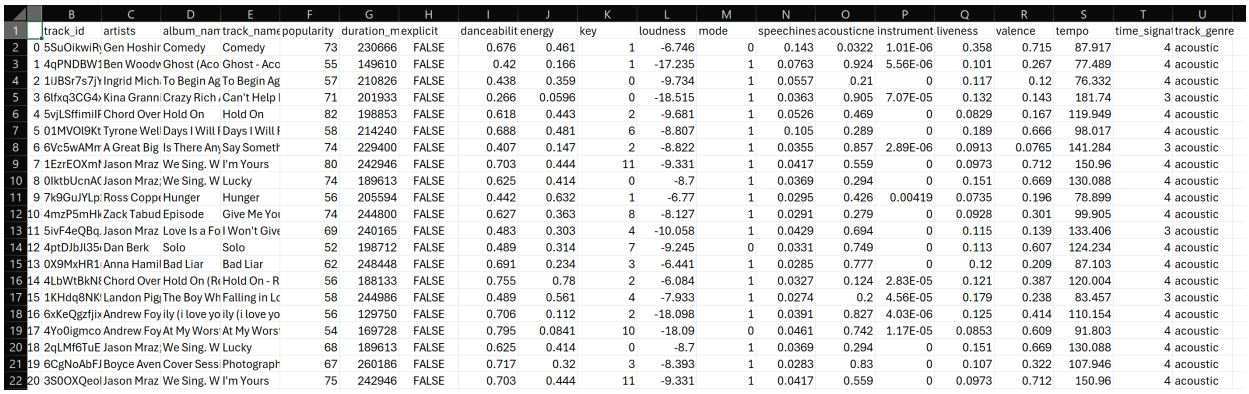
3. Dataset and Preprocessing

3.1 Source data

The catalog comes from a Spotify track-metadata dataset downloaded from Kaggle and stored at `data/raw/dataset.csv`. The spreadsheet view of the file (Figures 1 and 2) shows 114,000 data rows with one row per track-and-genre listing, an unnamed leading index column and the columns summarised below. The dataset provides **track metadata only**; it has no information about which users listened to which tracks.

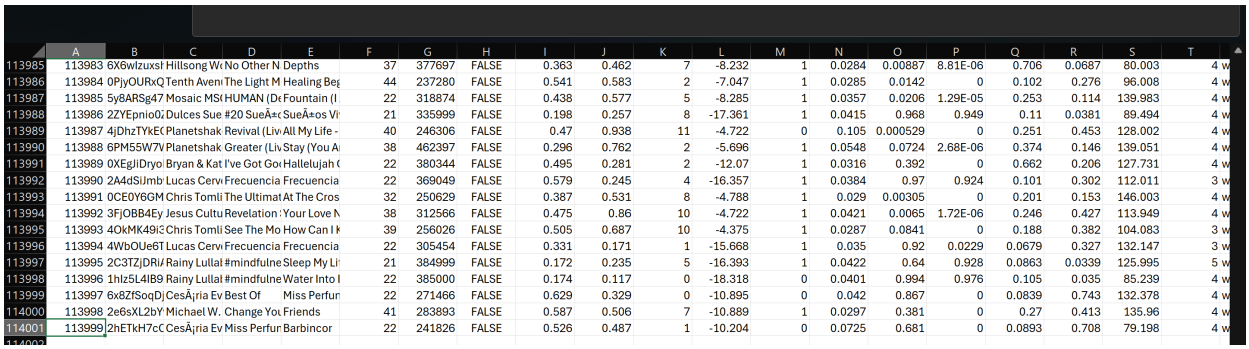
Column	Type	Role in this project
track_id	text	Unique key; interaction-matrix column identity
track_name, artists, album_name	text	Display fields; artists feed the TF-IDF artist features
track_genre	text	One-hot content feature; genre-mode filter; interaction generator
popularity	integer 0-100	Display, tie-breaker, sampling weight, genre-mode content score, popularity baseline

Column	Type	Role in this project
duration_ms, tempo, loudness, time_signature	numeric	Standardised content features
danceability, energy, speechiness, acousticness, instrumentalness, liveness, valence	numeric 0-1	Standardised content features
explicit, key, mode	boolean / integer	Present in the file; not used by the content engine



	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1	track_id	artists	album_nam	track_name	popularity	duration_m	explicit	danceabilit	energy	key	loudness	mode	speechines	acousticne	instrument	liveness	valence	tempo	time_signa	track_genre
2	0	5SuOikwiR	Gen Hoshir	Comedy	73	230666	FALSE	0.676	0.461	1	-6.746	0	0.143	0.0322	1.01E-06	0.358	0.715	87.917	4	acoustic
3	1	4qPNDBW1	Ben Woodv	Ghost (Aco	55	149610	FALSE	0.42	0.166	1	-17.235	1	0.0763	0.924	5.56E-06	0.101	0.267	77.489	4	acoustic
4	2	1iJBSr7s7j	Ingrid Mich	To Begin Ag	To Begin Ag	57	210826	FALSE	0.438	0.359	0	-9.734	1	0.0557	0.21	0	0.117	0.12	76.332	4	acoustic
5	3	6lfxq3CG4	Kina Grann	Crazy Rich	Can't Help	71	201933	FALSE	0.266	0.0596	0	-18.515	1	0.0363	0.905	7.07E-05	0.132	0.143	181.74	3	acoustic
6	4	5vjLSffimil	Chord Over	Hold On	Hold On	82	198853	FALSE	0.618	0.443	2	-9.681	1	0.0526	0.469	0	0.0829	0.167	119.949	4	acoustic
7	5	01MVOl9KI	Tyrone Wel	Days I Will	Days I Will	58	214240	FALSE	0.688	0.481	6	-8.807	1	0.105	0.289	0	0.189	0.666	98.017	4	acoustic
8	6	6Vc5wAMm	A Great Big	Is There Any	Say Somet	74	229400	FALSE	0.407	0.147	2	-8.822	1	0.0355	0.857	2.89E-06	0.0913	0.0765	141.284	3	acoustic
9	7	1EzrEOXmI	Jason Mraz	We Sing. W	I'm Yours	80	242946	FALSE	0.703	0.444	11	-9.331	1	0.0417	0.559	0	0.0973	0.712	150.96	4	acoustic
10	8	0IktbUcnA	Jason Mraz	We Sing. W	Lucky	74	189613	FALSE	0.625	0.414	0	-8.7	1	0.0369	0.294	0	0.151	0.669	130.088	4	acoustic
11	9	7k9GuJYLp	Ross Copp	Hunger	Hunger	56	205594	FALSE	0.442	0.632	1	-6.77	1	0.0295	0.426	0.00419	0.0735	0.196	78.899	4	acoustic
12	10	4mzP5mHk	Zack Tabud	Episode	Give Me Yo	74	244800	FALSE	0.627	0.363	8	-8.127	1	0.0291	0.279	0	0.0928	0.301	99.905	4	acoustic
13	11	5ivF4eQBq	Jason Mraz	Love Is a Fo	I Won't Give	69	240165	FALSE	0.483	0.303	4	-10.058	1	0.0429	0.694	0	0.115	0.139	133.406	3	acoustic
14	12	4ptDJbJl35	Dan Berk	Solo	Solo	52	198712	FALSE	0.489	0.314	7	-9.245	0	0.0331	0.749	0	0.113	0.607	124.234	4	acoustic
15	13	0X9MxHR1	Anna Hamil	Bad Liar	Bad Liar	62	248448	FALSE	0.691	0.234	3	-6.441	1	0.0285	0.777	0	0.12	0.209	87.103	4	acoustic
16	14	4LbWtBkN	Chord Over	Hold On (R	Hold On - R	56	188133	FALSE	0.755	0.78	2	-6.084	1	0.0327	0.124	2.83E-05	0.121	0.387	120.004	4	acoustic
17	15	1KHdq8NK	Landon Pig	The Boy Wh	Falling in Lo	58	244986	FALSE	0.489	0.561	4	-7.933	1	0.0274	0.2	4.56E-05	0.179	0.238	83.457	3	acoustic
18	16	6xKeQgzfji	Andrew Foy	ily (i love yo	ily (i love yo	56	129750	FALSE	0.706	0.112	2	-18.098	1	0.0391	0.827	4.03E-06	0.125	0.414	110.154	4	acoustic
19	17	4Yo0igmco	Andrew Foy	At My Wors	At My Wors	54	169728	FALSE	0.795	0.0841	10	-18.09	0	0.0461	0.742	1.17E-05	0.0853	0.609	91.803	4	acoustic
20	18	2qLMf6TuE	Jason Mraz	We Sing. W	Lucky	68	189613	FALSE	0.625	0.414	0	-8.7	1	0.0369	0.294	0	0.151	0.669	130.088	4	acoustic
21	19	6CgNoAbFJ	Boyce Aven	Cover Sess	Photograph	67	260186	FALSE	0.717	0.32	3	-8.393	1	0.0283	0.83	0	0.107	0.322	107.946	4	acoustic
22	20	3S0OXQeol	Jason Mraz	We Sing. W	I'm Yours	75	242946	FALSE	0.703	0.444	11	-9.331	1	0.0417	0.559	0	0.0973	0.712	150.96	4	acoustic

Figure 1. *First rows of the raw dataset (dataset.csv) opened in a spreadsheet, showing identifiers, metadata, audio features and the track_genre column.*



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
113985	113983	6X6wlzuxs	Hillsong W	No Other N	Depths	37	377697	FALSE	0.363	0.462	7	-8.232	1	0.0284	0.00887	8.81E-06	0.706	0.0687	80.003	4 w
113986	113984	0PjyOURxQ	Tenth Aven	The Light M	Healing Beg	44	237280	FALSE	0.541	0.583	2	-7.047	1	0.0285	0.0142	0	0.102	0.276	96.008	4 w
113987	113985	5y8ARSg47	Mosaic MS(	HUMAN (De	Fountain (I	22	318874	FALSE	0.438	0.577	5	-8.285	1	0.0357	0.0206	1.29E-05	0.253	0.114	139.983	4 w
113988	113986	2ZYEpnio0	Dulces Sue	#20 SueÃ±o	SueÃ±os Vi	21	335999	FALSE	0.198	0.257	8	-17.361	1	0.0415	0.968	0.949	0.11	0.0381	89.494	4 w
113989	113987	4jDhzTYkE	Planetshak	Revival (Liv	All My Life -	40	246306	FALSE	0.47	0.938	11	-4.722	0	0.105	0.000529	0	0.251	0.453	128.002	4 w
113990	113988	6PM55W7V	Planetshak	Greater (Liv	Stay (You A	38	462397	FALSE	0.296	0.762	2	-5.696	1	0.0548	0.0724	2.68E-06	0.374	0.146	139.051	4 w
113991	113989	0XEgJiDryo	Bryan & Kat	I've Got Go	Hallelujah (	22	380344	FALSE	0.495	0.281	2	-12.07	1	0.0316	0.392	0	0.662	0.206	127.731	4 w
113992	113990	2A4dSiJmb	Lucas Cerv	Frecuencia	Frecuencia	22	369049	FALSE	0.579	0.245	4	-16.357	1	0.0384	0.97	0.924	0.101	0.302	112.011	3 w
113993	113991	0CE0Y6GM	Chris Tomli	The Ultimat	At The Cros	32	250629	FALSE	0.387	0.531	8	-4.788	1	0.029	0.00305	0	0.201	0.153	146.003	4 w
113994	113992	3FjOBB4Ey	Jesus Cultu	Revelation :	Your Love N	38	312566	FALSE	0.475	0.86	10	-4.722	1	0.0421	0.0065	1.72E-06	0.246	0.427	113.949	4 w
113995	113993	4OkMK49i3	Chris Tomli	See The Mo	How Can I k	39	256026	FALSE	0.505	0.687	10	-4.375	1	0.0287	0.0841	0	0.188	0.382	104.083	3 w
113996	113994	4WbOUe6T	Lucas Cerv	Frecuencia	Frecuencia	22	305454	FALSE	0.331	0.171	1	-15.668	1	0.035	0.92	0.0229	0.0679	0.327	132.147	3 w
113997	113995	2C3TZjDRi	Rainy Lullal	#mindfulne	Sleep My Li	21	384999	FALSE	0.172	0.235	5	-16.393	1	0.0422	0.64	0.928	0.0863	0.0339	125.995	5 w
113998	113996	1hIz5L4IB9	Rainy Lullal	#mindfulne	Water Into	22	385000	FALSE	0.174	0.117	0	-18.318	0	0.0401	0.994	0.976	0.105	0.035	85.239	4 w
113999	113997	6x8ZfSoqDj	CesÃ¡ria Ev	Best Of	Miss Perfun	22	271466	FALSE	0.629	0.329	0	-10.895	0	0.042	0.867	0	0.0839	0.743	132.378	4 w
114000	113998	2e6sXL2bY	Michael W.	Change You	Friends	41	283893	FALSE	0.587	0.506	7	-10.889	1	0.0297	0.381	0	0.27	0.413	135.96	4 w
114001	113999	2hETkH7cC	CesÃ¡ria Ev	Miss Perfur	Barbincor	22	241826	FALSE	0.526	0.487	1	-10.204	0	0.0725	0.681	0	0.0893	0.708	79.198	4 w

Figure 2. *Last rows of the raw dataset; the final record sits on spreadsheet row 114,001 (header on row 1), i.e. 114,000 raw records.*

3.2 Preprocessing pipeline

Cleaning is implemented in `recommender/preprocessing.py` by `clean_dataset(source, destination)` and executed by `scripts/prepare_data.py`. The routine produces one useful row per track and writes the result as Parquet.

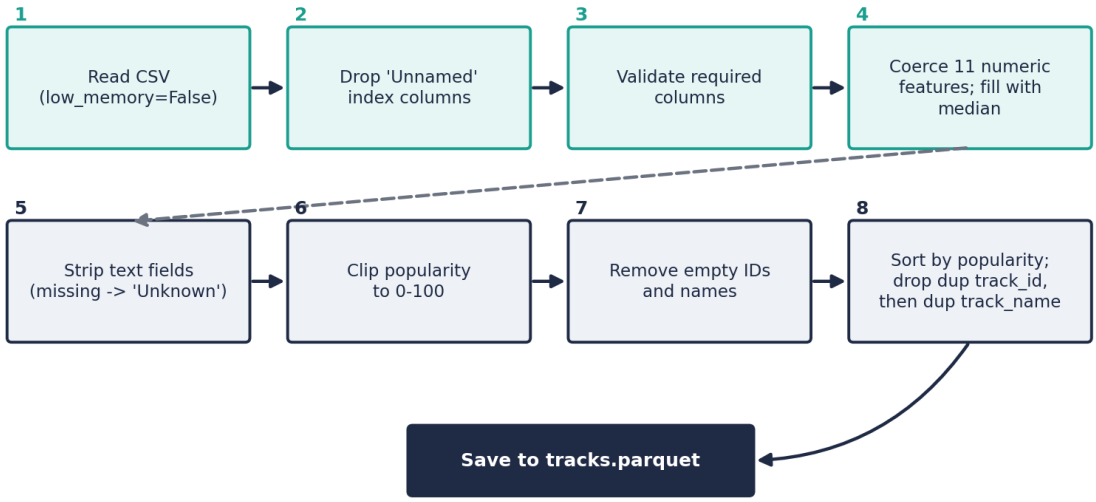


Figure 3. *Preprocessing pipeline implemented in clean_dataset().*

Step	Implementation detail	Reason
Drop index columns	Columns whose names start with 'Unnamed' are removed	Remove the export artefact
Validate schema	Required: track_id, track_name, artists, popularity, duration_ms, track_genre; raises ValueError if any is missing	Fail fast with a clear message
Numeric coercion	Eleven feature columns converted with pd.to_numeric(errors='coerce'); missing columns default to 0.0; NaNs filled with the column median (0.0 if all missing)	Robust, outlier-tolerant imputation
Text cleaning	track_id, track_name, artists, album_name, track_genre: fillna('Unknown'), cast to str, strip whitespace	Consistent keys and labels
Popularity range	Coerce, fill 0, clip to [0, 100]	Guarantee a valid scale
Invalid rows	Rows with empty track_id or track_name removed	No unusable records
De-duplication	Sort by popularity (descending), drop duplicate track_id, then drop duplicate track_name, reset index	Keep the most popular version of each track; one row per ID and name
Persistence	Create the output folder and write tracks.parquet (index=False)	Fast, typed reload with PyArrow

Note that de-duplicating by track name keeps a single record per name, so different songs that happen to share a title are collapsed. This keeps the catalog compact and avoids near-duplicate recommendations, but it can drop legitimate tracks (see Section 9).

3.3 Content feature set

The content engine uses eleven numeric columns (danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo, duration_ms, time_signature), the track genre and the artist string. The columns key, mode and explicit exist in the file but are not part of the model.

4. System Architecture

The system is organised as an offline build stage and an online serving stage. The offline stage cleans data, generates synthetic interactions and trains the two engines, saving them as Joblib artifacts. The online stage loads the artifacts once (cached by Streamlit) and answers each request by scoring, normalising, blending and ranking candidates.

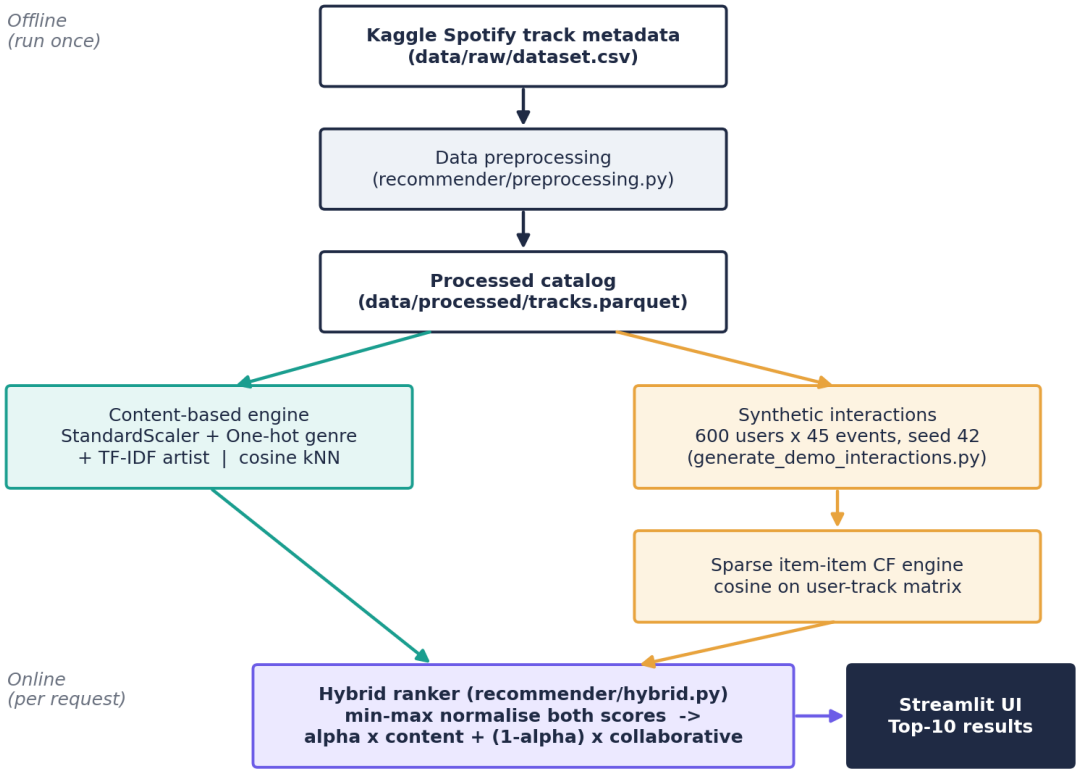


Figure 4. *End-to-end architecture of the hybrid music recommender.*

4.1 Technology stack

```
pandas>=2.0
numpy>=1.24
scipy>=1.10
scikit-learn>=1.3
streamlit>=1.30
joblib>=1.3
pyarrow>=14.0
matplotlib>=3.7
pytest>=7.4
```

Figure 5. *requirements.txt of the project.*

Library	Minimum version	Purpose
pandas	2.0	Tabular data handling, score Series
numpy	1.24	Numerical operations, random generation
scipy	1.10	Sparse matrices (CSR), saving .npz

Library	Minimum version	Purpose
scikit-learn	1.3	ColumnTransformer, scalers, encoders, TF-IDF, NearestNeighbors
streamlit	1.30	Web interface
joblib	1.3	Persisting trained engines
pyarrow	14.0	Parquet read/write
matplotlib	3.7	Evaluation chart
pytest	7.4	Basic checks

4.2 Project structure

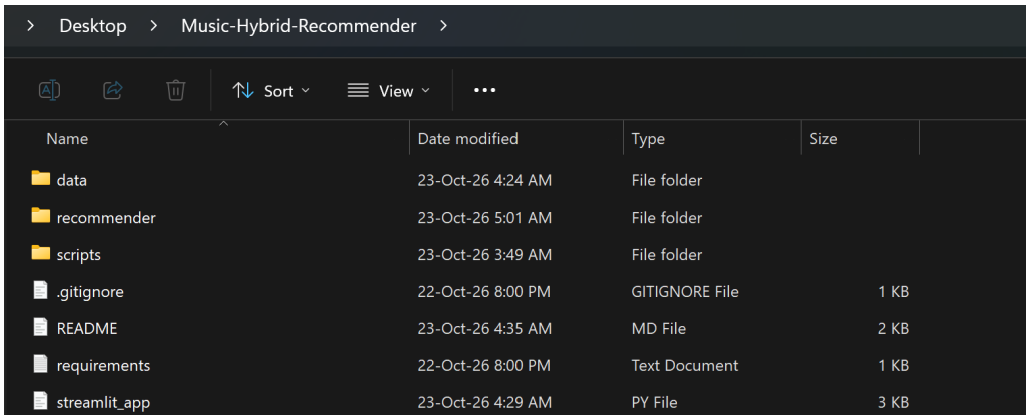


Figure 6. *Project root folder (Desktop/Music-Hybrid-Recommender) showing data, recommender, scripts, README, requirements and the Streamlit app.*

```
Music-Hybrid-Recommender/
|-- data/
|   |-- raw/dataset.csv
|   |-- processed/tracks.parquet
|   `-- interactions/synthetic_interactions.npz
|-- models/                      (created by the training script)
|   |-- content.joblib
|   `-- collaborative.joblib
|-- recommender/
|   |-- preprocessing.py   content_based.py   collaborative.py
|   `-- hybrid.py          evaluation.py
|-- scripts/
|   |-- prepare_data.py    generate_demo_interactions.py
|   |-- train_models.py    optimize_hybrid.py    evaluate_models.py
|-- streamlit_app.py   requirements.txt   README.md
```

5. Methodology

5.1 Content-based engine

Implemented in `ContentRecommender` (`recommender/content_based.py`). The constructor resets the catalog index, builds a track-ID-to-row lookup and fits a `ColumnTransformer` with three branches: **StandardScaler** on the numeric features, **OneHotEncoder(handle_unknown='ignore')** on track_genre, and **TfidfVectorizer** with token pattern `[^,]+` on the artists column so that each comma-separated artist is one token. The transformed matrix is stored as a SciPy CSR matrix and a brute-force cosine `NearestNeighbors` index is fitted on it.

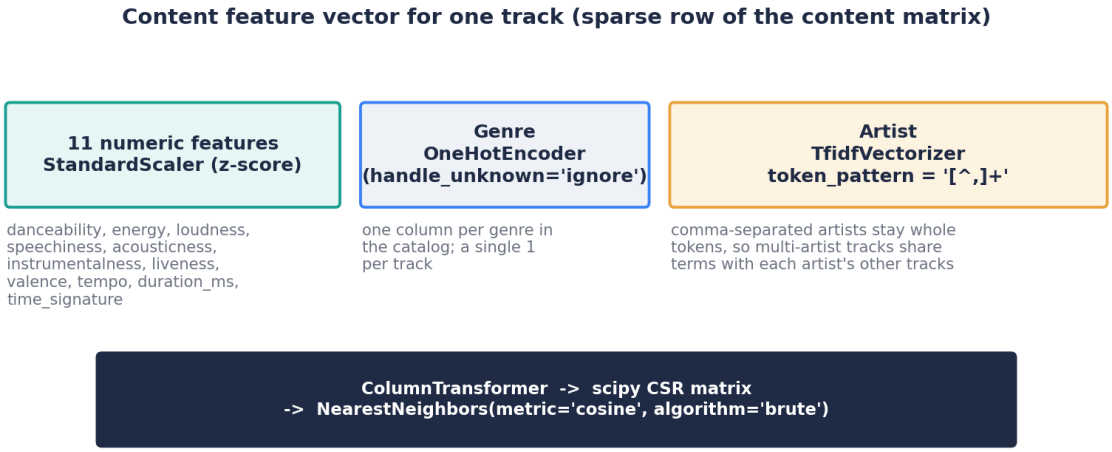


Figure 7. Composition of a track's content feature vector.

Scoring. `scores_for_track(track_id, candidates=None)` works in two modes. Without candidates, it queries the k-nearest-neighbour index for up to 301 neighbours and returns similarity = 1 - cosine distance, dropping the seed itself. With an explicit candidate list, it computes cosine similarity directly between the seed row and each candidate row (dot product divided by the row norms, with zero-norm rows scoring 0). The second mode lets the hybrid engine score tracks that the collaborative engine surfaced but the neighbour search did not.

5.2 Synthetic interaction generation

Because the metadata has no listening histories, `simulate_interactions(data, users=600, per_user=45, seed=42)` in `recommender/collaborative.py` produces reproducible, genre-biased events. Each user chooses three favourite genres without replacement, pools all tracks in those genres, assigns each pooled track a sampling weight of popularity + 5, and draws 45 distinct tracks with probability proportional to that weight. The result is a binary CSR matrix of shape (600, number of tracks) with float32 values, saved as `synthetic_interactions.npz`.

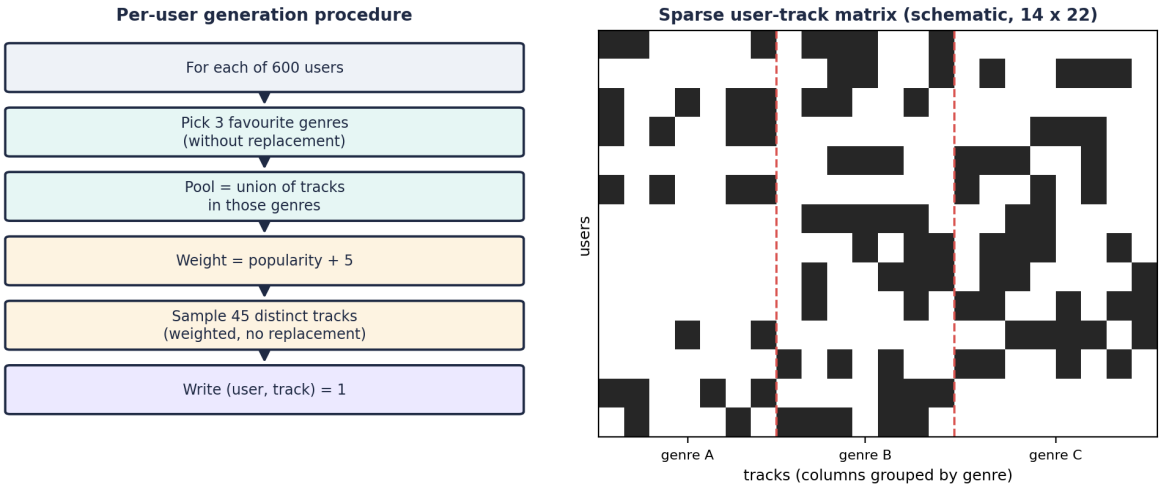


Figure 8. Left: per-user synthetic interaction procedure. Right: schematic of the resulting sparse matrix, where users cluster on a few genres (illustrative drawing, not real data).

Parameter	Value	Meaning
users	600	Number of simulated users (matrix rows)

Parameter	Value	Meaning
per_user	45	Interactions drawn per user (capped by pool size)
favourite genres	3	Genres per user; creates genre-biased co-occurrence
popularity offset	+5	Gives zero-popularity tracks a small non-zero chance
seed	42	Makes the simulation reproducible

This design is deliberately simple and transparent. It also means the collaborative engine can only learn what the generator encodes: tracks that share genres and tracks that are popular. This is why all collaborative results are described as demonstrations, never as evidence of real listener behaviour.

5.3 Item-item collaborative filtering engine

Implemented in `ItemCollaborativeFilter`. The constructor stores the track IDs, builds ID-to-column maps, converts the matrix to CSR, computes per-item popularity (column sums unless supplied) and per-item L2 norms (square root of the column-wise sum of squares).

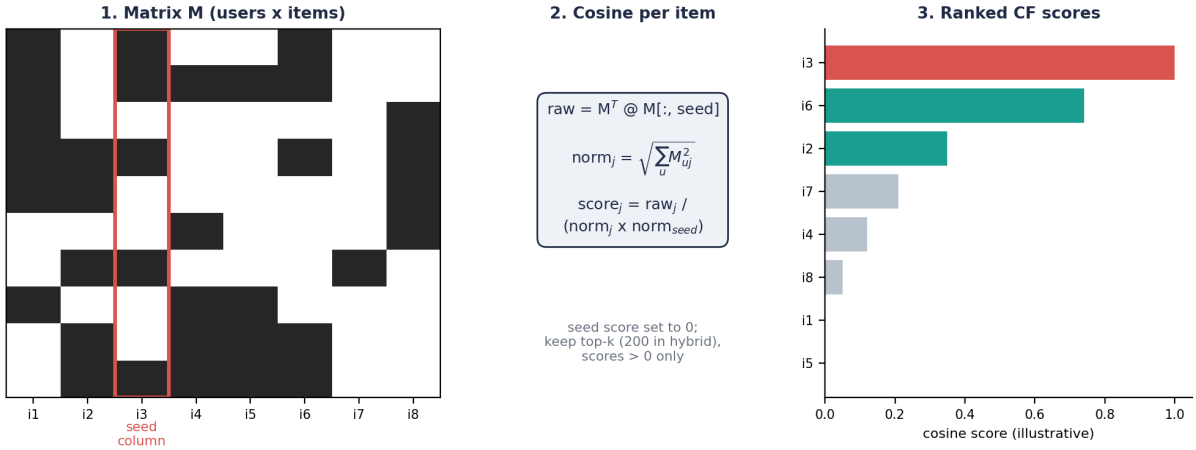


Figure 9. *Item-item scoring for one seed track: select its column, take cosine similarity with every item column, rank the results (schematic with illustrative values).*

Scoring for a seed. `scores_for_track(track_id, top_k=500)` computes the raw co-occurrence vector M^T times the seed column, divides it by the product of norms (only where the denominator is positive), zeroes the seed's own score, selects the top-k indices with `argpartition`, maps them back to track IDs, keeps only positive scores and sorts them in descending order. The hybrid engine requests top_k = 200.

Genre scores. `genre_scores(columns)` returns the global item interaction count for the supplied tracks. As the code comment notes, no individual user is fabricated; global synthetic-interaction support is used.

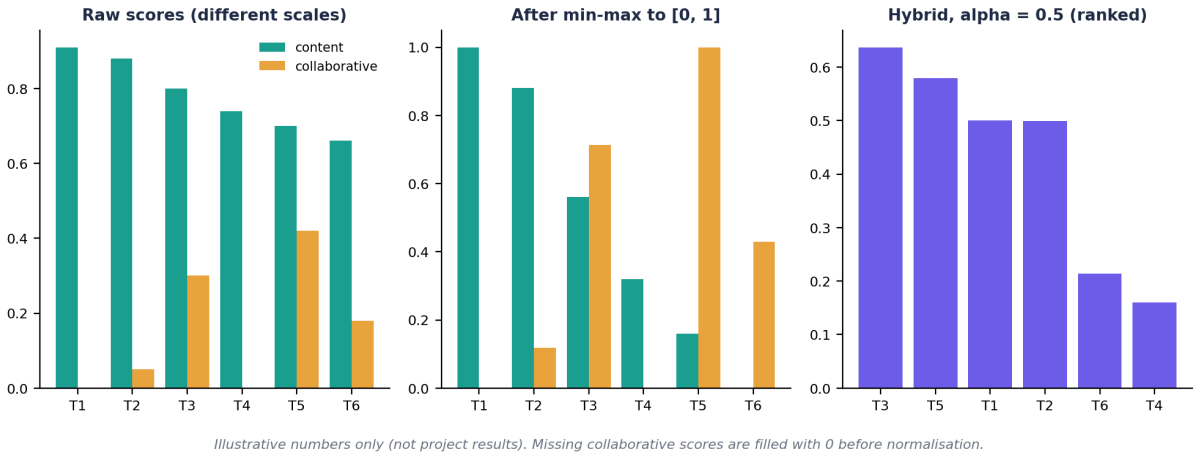
5.4 Hybrid ranking

The hybrid engine (`HybridRecommender` in `recommender/hybrid.py`) normalises each score vector with min-max scaling over the candidate set and then blends them:

HybridScore = alpha x NormContent + (1 - alpha) x NormCollaborative

Min-max normalisation maps the lowest candidate to 0 and the highest to 1; if all values are equal (zero span) it returns zeros. Missing collaborative scores are treated as 0 *before*

normalisation. The final ranking sorts by hybrid score and then popularity (both descending), drops duplicate track IDs and returns the top *n* (default 10) with the component scores attached.



Illustrative numbers only (not project results). Missing collaborative scores are filled with 0 before normalisation.

Figure 10. *Worked illustration of the hybrid blend: raw scores on different scales, min-max normalised scores and the ranked hybrid score for alpha = 0.5. Values are illustrative.*

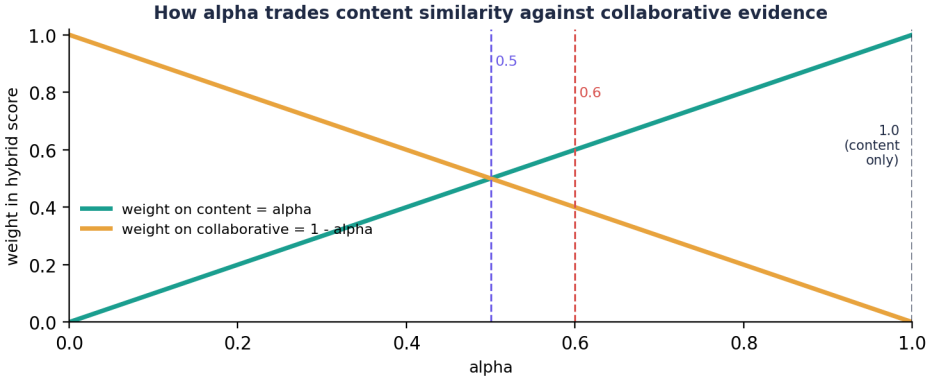


Figure 11. *Weights applied to the two engines as alpha varies, with the alpha values used in the code marked.*

5.5 Recommend by Song

`recommend_by_song(track_id, top_n=10, alpha=0.6)` first obtains an initial content score set (up to 301 neighbours) and an initial collaborative score set (top 200). If the collaborative set is empty or sums to zero, it falls back to content-only ranking by calling the ranker with alpha = 1.0. Otherwise it forms the candidate set as the union of both index sets, re-scores every candidate with content cosine similarity so that the two score vectors cover the same items, fills missing collaborative values with 0, and ranks while excluding the seed track.

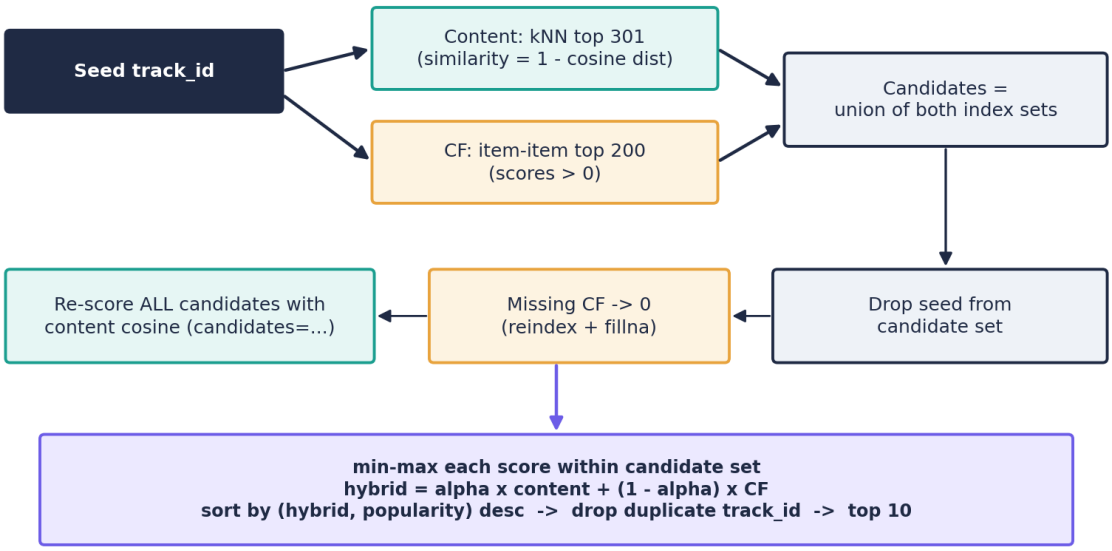


Figure 12. *Candidate construction and scoring in song mode.*

5.6 Recommend by Genre

`recommend_by_genre(genre, top_n=10, alpha=0.5)` raises a KeyError for unknown genres, collects every track in the selected genre as candidates, uses track popularity as a transparent content-side score and the global synthetic interaction support as the collaborative-side score, and then applies the same normalise-and-blend ranking. The ranker also enforces the genre filter, so results never leave the selected genre.

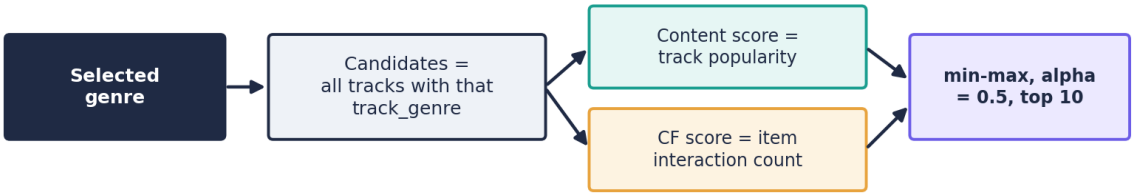


Figure 13. *Candidate and score construction in genre mode.*

Aspect	Recommend by Song	Recommend by Genre
Input	track_id of a selected song	Genre label
Candidates	Union of content top-301 and CF top-200	All tracks of the genre
Content score	Cosine similarity to the seed	Track popularity
Collaborative score	Item-item cosine to the seed	Global item interaction count
Default alpha (code)	0.6	0.5
Exclusions	Seed track removed	Only the selected genre allowed
Fallback	Content-only (alpha = 1.0) if CF is empty	None needed

6. Implementation

6.1 Module responsibilities

File	Responsibility
recommender/preprocessing.py	Cleaning, validation, imputation, de-duplication, Parquet save and load_processed()
recommender/content_based.py	ContentRecommender: feature pipeline, cosine kNN, similar-track queries
recommender/collaborative.py	ItemCollaborativeFilter and simulate_interactions()
recommender/hybrid.py	minmax() helper and HybridRecommender (song and genre modes)
recommender/evaluation.py	Precision, Recall, MAP, NDCG, catalogue coverage, diversity
scripts/prepare_data.py	Runs clean_dataset on data/raw/dataset.csv
scripts/generate_demo_interactions.py	Generates and saves synthetic interactions, printing a notice that they are not real listener data
scripts/train_models.py	Builds ContentRecommender and ItemCollaborativeFilter, saves both with joblib (compress=3)
scripts/optimize_hybrid.py	Grid search of alpha on held-out synthetic events
scripts/evaluate_models.py	Compares content, collaborative, hybrid and popularity baseline; saves CSV and chart
streamlit_app.py	Interactive interface

6.2 Execution workflow

The scripts must be executed in order because each produces the input of the next. The commands (Windows PowerShell, from the project root) are listed in Appendix A.

6.3 Training and persistence

`scripts/train_models.py` loads the processed catalog, instantiates the content engine, instantiates the collaborative engine from the track IDs and the saved sparse matrix, and writes `models/content.joblib` and `models/collaborative.joblib`. It creates the `models/` folder if necessary. The README notes that the first build can take a few minutes because the full catalog is processed. The optimised alpha is not written into these artifacts; the application must be configured to use it.

6.4 Streamlit application

`streamlit_app.py` sets a dark theme, loads the catalog and engines through an `st.cache_resource` function and shows a clear error with instructions if artifacts are missing. The page header reports the number of tracks and genres. A radio button selects one input mode: a searchable select box of ‘track - artist [genre]’ labels for song mode, or a sorted genre select box for genre mode. Pressing **Recommend Songs** calls the hybrid engine, renames the columns for display and shows a table with Song, Artist, Genre, Popularity, Content score, Collaborative score and Hybrid score (three decimals). A caption explains the normalisation, the zero fill for missing collaborative values, result uniqueness, seed exclusion and genre confinement.

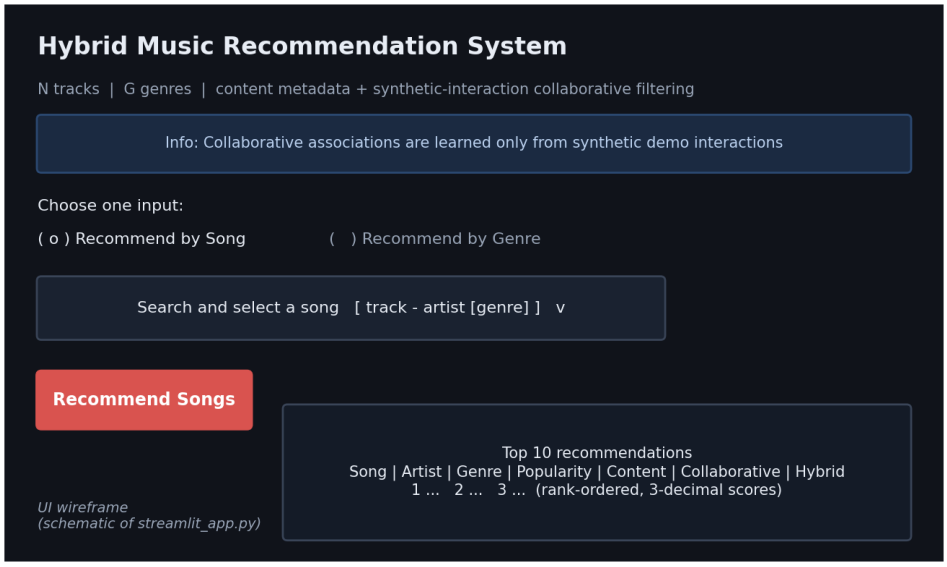


Figure 14. *Schematic wireframe of the Streamlit interface (drawn from the layout defined in streamlit_app.py; not a screenshot).*

6.5 Hyper-parameters and defaults

Component	Parameter	Value
Content engine	Neighbours returned (kneighbors)	min(catalog size, 301)
Content engine	Metric / algorithm	cosine / brute
Collaborative engine	Default top_k / hybrid top_k	500 / 200
Hybrid (song mode)	alpha default in code	0.6
Hybrid (genre mode)	alpha default in code	0.5
Hybrid (ranker)	Internal default alpha, final top_n	0.5, 10
Hybrid fallback	alpha when CF is empty	1.0
Synthetic data	users / per_user / seed	600 / 45 / 42
evaluate_models.py	Users evaluated, fixed alpha	up to 200, 0.5
optimize_hybrid.py	Users evaluated, alpha grid	up to 500, 0.0 to 1.0 step 0.1 (11 values)

7. Evaluation Methodology

7.1 Metrics

Metrics are implemented in `recommender/evaluation.py`. For each recommendation list the top K items are compared with a set of relevant (held-out) items.

Metric	Definition as implemented
Precision@K	Hits in the top K divided by K, averaged over users
Recall@K	Hits in the top K divided by the number of relevant items, averaged over users
MAP@K	Mean over users of average precision: sum of precision at each hit position divided by min(K, number of relevant items)

Metric	Definition as implemented
NDCG@K	DCG with discount 1 / log2(rank + 1) divided by the ideal DCG for the number of relevant items (up to K)
Catalogue coverage	Number of distinct recommended tracks across all users divided by the catalog size
Diversity	Mean of 1 - similarity over all pairs within each recommendation list (helper function)

7.2 Held-out protocol

`evaluate_models.py` removes one random interaction per user from the synthetic matrix, rebuilds the collaborative engine on the reduced matrix so that training never sees the held-out cell, and uses the user's first remaining track as the seed. For up to 200 users it produces top-10 lists from four systems: **content-only**, **collaborative-only**, **hybrid** (alpha = 0.5) and a **popularity baseline** (the ten most popular tracks in the catalog, identical for every user). The held-out track is the single relevant item. Results are written to `models/evaluation_metrics.csv` and plotted as a bar chart in `models/evaluation_chart.png`.

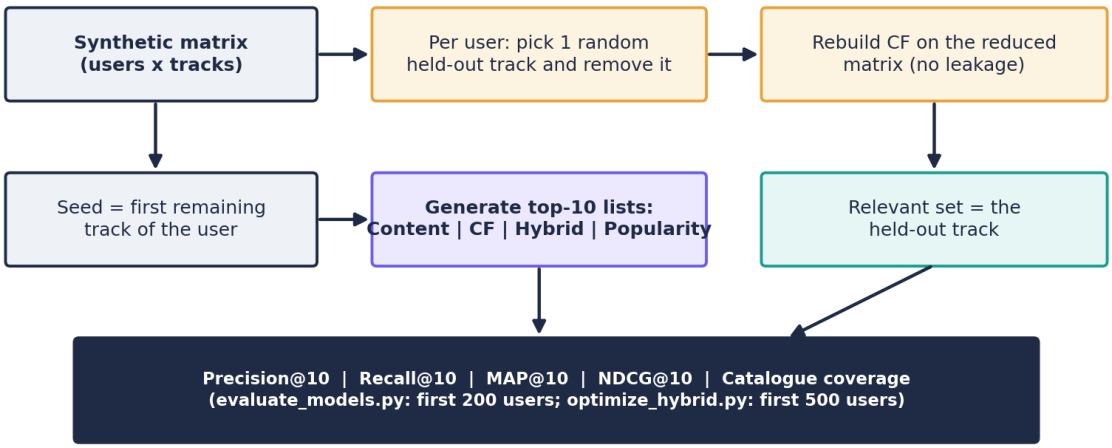


Figure 15. *Held-out evaluation protocol shared by the evaluation and optimisation scripts.*

Interpreting the numbers. Because exactly one item is held out per user, Precision@10 can be at most 0.1, Recall@10 equals the hit rate, and MAP@10 and NDCG@10 depend only on the rank of that single item (NDCG = 1 / log2(rank + 1) when it is found). Absolute values will therefore look small, and comparisons between systems are more meaningful than the values themselves.

7.3 Alpha tuning

`optimize_hybrid.py` removes one interaction per user (skipping users with none), builds the collaborative engine on the remaining data, and for up to 500 users caches the content and collaborative candidate scores for the seed track. It then loops over eleven alpha values from 0.0 to 1.0 in steps of 0.1, recomputes the blended top-10 list for each user, computes Precision@10, Recall@10 and NDCG@10, prints one line per alpha and reports the alpha with the highest NDCG@10. Cached scores make the sweep cheap, because only the blend is recomputed.

7.4 Reported results

Running `python scripts\evaluate_models.py` and `python scripts\optimize_hybrid.py` on the full catalog produces the comparison table, the bar chart and the optimal alpha for the user's machine. These outputs depend on the generated synthetic data and were not part of the files supplied for this report, so no numeric results are quoted here. The table below lists where each result is stored so that it can be appended to this report.

Output	Location	Content
Comparison metrics	models/evaluation_metrics.csv	Precision@10, Recall@10, MAP@10, NDCG@10, catalogue coverage for four systems
Comparison chart	models/evaluation_chart.png	Bar chart titled 'Held-out synthetic interaction metrics'
Alpha sweep	Console output of optimize_hybrid.py	Precision, Recall and NDCG for alpha 0.0 to 1.0 and the best alpha

Reading the results correctly. The synthetic generator ties users to genres and favours popular tracks. A collaborative or hybrid system that scores well is partly recovering that generator, and the popularity baseline is expected to benefit from the popularity-weighted sampling. The comparison is therefore a sanity check of the pipeline, not evidence about real music taste.

8. Discussion

8.1 Design strengths

- **Scalable by construction.** The system never builds a catalog-wide dense similarity matrix; it computes similarity for one seed against sparse matrices.
- **Candidate-set alignment.** Content scores are recomputed for all collaborative candidates, so both score vectors cover the same tracks before normalisation.
- **Graceful degradation.** If collaborative evidence is empty, the system falls back to pure content ranking.
- **Transparent output.** Component scores are shown next to the hybrid score so that users and reviewers can see why a track ranked highly.
- **Modular code.** Preprocessing, engines, blending, evaluation, scripts and UI are separate and individually testable.
- **Honest framing.** Synthetic data is labelled as such in code messages, UI text and documentation.

8.2 Observations on the current implementation

- **Alpha defaults differ.** The README states that the hybrid recommender uses alpha = 0.5 by default, while song-mode recommendations in the code default to 0.6 (genre mode uses 0.5). The Streamlit app does not pass alpha, so these defaults apply.
- **Tuned alpha is not applied automatically.** optimize_hybrid.py only prints the best value, so the app must be edited if the tuned value is to be used.
- **Min-max normalisation is relative.** Scores are scaled within each candidate set, so a hybrid score of 1.0 means best among candidates, not an absolute match.

- **Genre mode is weakly personalised.** With popularity on both sides, the ranking essentially orders the genre by popularity and interaction counts.

9. Limitations

- Collaborative filtering is trained on **synthetic, genre-biased interactions**, not real listener histories.
- Evaluation measures agreement with held-out **synthetic** events and does not establish real-world recommendation performance.
- Only one relevant item per user is held out, which makes metrics coarse and bounds Precision@10 at 0.1.
- Recommendations depend on the supplied catalog and available metadata; keys, modes and explicit flags are unused.
- De-duplicating by track name may remove distinct songs that share a title.
- No cold-start handling for tracks absent from the interaction matrix beyond content similarity.
- No user profiles, sessions, or context (time, mood, activity) are modelled.

10. Future Work

- Replace synthetic interactions with real listening logs (for example from consenting users or an open listening-history dataset) and re-tune alpha.
- Add matrix-factorisation or neural embedding models (ALS, implicit-feedback models) alongside item-item filtering.
- Allow multi-song or playlist seeds and simple user profiles.
- Store the optimised alpha with the saved models and expose it as a slider in the interface for interactive exploration.
- Evaluate diversity, novelty and serendipity (the diversity helper already exists) and add re-ranking for variety.
- Expand automated tests with pytest for preprocessing, scoring, normalisation edge cases and mode constraints.
- Add audio previews and cover art, and deploy the app (for example on Streamlit Community Cloud or a container).

11. Conclusion

The project delivers a complete, reproducible hybrid music recommender: a cleaned Parquet catalog, a standardised-feature and TF-IDF content engine, a sparse item-item collaborative engine trained on clearly labelled synthetic interactions, a normalised weighted hybrid ranker with song and genre modes, evaluation and tuning scripts, and a transparent Streamlit interface. Its architecture avoids dense similarity matrices, aligns candidate sets before blending and degrades gracefully when collaborative evidence is missing. The principal limitation, the absence of real interaction data, is stated openly, and the codebase is organised so that real listening logs can replace the synthetic generator without changing the rest of the pipeline.

References

Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331-370.

Jarvelin, K., and Kekalainen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4), 422-446.

Salton, G., and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5), 513-523.

Sarwar, B., Karypis, G., Konstan, J., and Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. *Proceedings of the 10th International Conference on World Wide Web*, 285-295.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Spotify Tracks Dataset, Kaggle (track metadata and audio features used as the catalog).

Appendix A. Setup and Run Commands

Windows PowerShell, run from the project root. The raw dataset must exist at `data/raw/dataset.csv`.

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts\prepare_data.py
python scripts\generate_demo_interactions.py
python scripts\train_models.py
streamlit run streamlit_app.py
```

If PowerShell blocks activation of the virtual environment:

```
Set-ExecutionPolicy -Scope Process Bypass
.\.venv\Scripts\Activate.ps1
```

Optional evaluation and tuning (after data preparation and training):

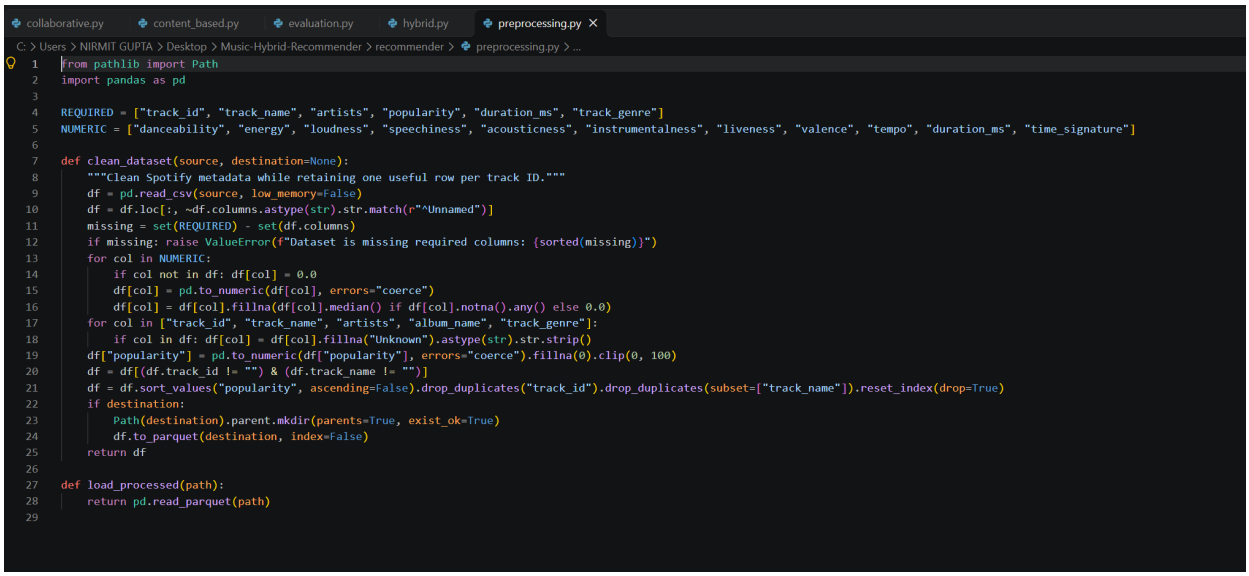
```
python scripts\optimize_hybrid.py
python scripts\evaluate_models.py
```

Symptom	Remedy
App reports model artifacts are unavailable	Run prepare_data.py, generate_demo_interactions.py and train_models.py in order
PowerShell blocks .venv activation	Set-ExecutionPolicy -Scope Process Bypass, then activate again
First training run is slow	Expected: the full catalog is processed
Evaluation fails	Make sure models/content.joblib and the synthetic interactions file exist

Appendix B. Source Code Listings (Screenshots)

The screenshots below show the implementation files as they appear in the editor.

B.1 recommender/preprocessing.py



B.2 recommender/collaborative.py

collaborative.py | content_based.py | evaluation.py | hybrid.py | preprocessing.py

C: > Users > NIRMIT GUPTA > Desktop > Music-Hybrid-Recommender > recommender > collaborative.py > ...

```python
import numpy as np
import pandas as pd
from scipy import sparse

class ItemCollaborativeFilter:
    """Sparse item-item CF trained only on labelled synthetic demonstration events."""
    def __init__(self, track_ids, matrix, popularity=None):
        self.track_ids = np.asarray(track_ids)
        self.id_to_col = {v:i for i,v in enumerate(self.track_ids)}
        self.col_to_id = {i: v for i, v in enumerate(self.track_ids)}
        self.matrix = matrix.tocsr()
        self.item_popularity = np.asarray(popularity if popularity is not None else self.matrix.sum(axis=0)).ravel()
        norms = np.sqrt(np.asarray(self.matrix.multiply(self.matrix).sum(axis=0)).ravel())
        self.norms = norms

    def scores_for_track(self, track_id, top_k=500):
        if track_id not in self.id_to_col: raise KeyError(f"Unknown track_id: {track_id}")
        col = self.id_to_col[track_id]
        raw = (self.matrix.T @ self.matrix[:, col]).toarray().ravel()
        denom = self.norms * self.norms[col]
        scores = np.divide(raw, denom, out=np.zeros_like(raw, dtype=float), where=denom > 0)
        scores[col] = 0
        inds = np.argpartition(scores, -min(top_k, len(scores)))[-min(top_k, len(scores)):]
        col_to_id = {v: k for k, v in self.id_to_col.items()}
        mapped_track_ids = [col_to_id[i] for i in inds]
        return pd.Series(scores[inds], index=mapped_track_ids)[lambda s: s > 0].sort_values(ascending=False)

    def genre_scores(self, columns):
        # No individual user is fabricated: global synthetic interaction support is used.
        return pd.Series(self.item_popularity[np.asarray(columns)], index=np.asarray(columns))

def simulate_interactions(data, users=600, per_user=45, seed=42):
    rng = np.random.default_rng(seed); n = len(data)
    genres = data.groupby("track_genre").indices
    rows=[]; cols=[]
    for user in range(users):
        fav = rng.choice(list(genres), size=min(3, len(genres)), replace=False)
        pool = np.unique(np.concatenate([genres[g] for g in fav]))
        pop = data.iloc[pool].popularity.to_numpy(float) + 5
        take = min(per_user, len(pool)); chosen = rng.choice(pool, size=take, replace=False, p=pop/pop.sum())
        rows.extend([user]*take); cols.extend(chosen.tolist())
    matrix = sparse.csr_matrix((np.ones(len(rows), dtype=np.float32), (rows, cols)), shape=(users, n))
    return matrix
```

B.3 recommender/content_based.py

collaborative.py | content_based.py | evaluation.py | hybrid.py | preprocessing.py

C: > Users > NIRMIT GUPTA > Desktop > Music-Hybrid-Recommender > recommender > content_based.py > ...

```python
import numpy as np
import pandas as pd
from scipy import sparse
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import NearestNeighbors
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from .preprocessing import NUMERIC

class ContentRecommender:
    def __init__(self, data):
        self.data = data.reset_index(drop=True).copy()
        self.id_to_index = pd.Series(self.data.index, index=self.data.track_id).to_dict()
        self.features = [x for x in NUMERIC if x in self.data]
        self.transformer = ColumnTransformer([
            ("numbers", StandardScaler(), self.features),
            ("genre", OneHotEncoder(handle_unknown="ignore"), ["track_genre"]),
            ("artist", TfidfVectorizer(token_pattern=r"[^,]+"), "artists"),
        ])
        self.matrix = sparse.csr_matrix(self.transformer.fit_transform(self.data))
        self.knn = NearestNeighbors(metric="cosine", algorithm="brute", n_jobs=-1).fit(self.matrix)

    def scores_for_track(self, track_id, candidates=None):
        if track_id not in self.id_to_index: raise KeyError(f"Unknown track_id: {track_id}")
        idx = self.id_to_index[track_id]
        if candidates is None:

            distances, inds = self.knn.kneighbors(self.matrix[idx], n_neighbors=min(len(self.data), 301))
            mapped_track_ids = self.data["track_id"].iloc[inds.ravel()].values
            return pd.Series(1 - distances.ravel(), index=mapped_track_ids).drop(track_id, errors="ignore")
        inds = np.asarray(candidates, dtype=int)
        sims = self.matrix[inds].dot(self.matrix[idx].T).toarray().ravel()
        norms = np.sqrt(self.matrix[inds].multiply(self.matrix[inds]).sum(axis=1)).A1 * np.sqrt(self.matrix[idx].multiply(self.matrix[idx]).sum())
        mapped_candidates = self.data["track_id"].iloc[inds].values
        return pd.Series(np.divide(sims, norms, out=np.zeros_like(sims), where=norms != 0), index=mapped_candidates)

    def recommend_similar_tracks(self, track_id, top_n=10):
        s = self.scores_for_track(track_id).sort_values(ascending=False).head(top_n)
        out = self.data.iloc[s.index][["track_id", "track_name", "artists", "track_genre", "popularity"]].copy()
        out["similarity_score"] = s.values
        return out.reset_index(drop=True)

```

B.4 recommender/hybrid.py

collaborative.py | content_based.py | evaluation.py | hybrid.py | preprocessing.py

C: > Users > NIRMIT GUPTA > Desktop > Music-Hybrid-Recommender > recommender > hybrid.py > ...

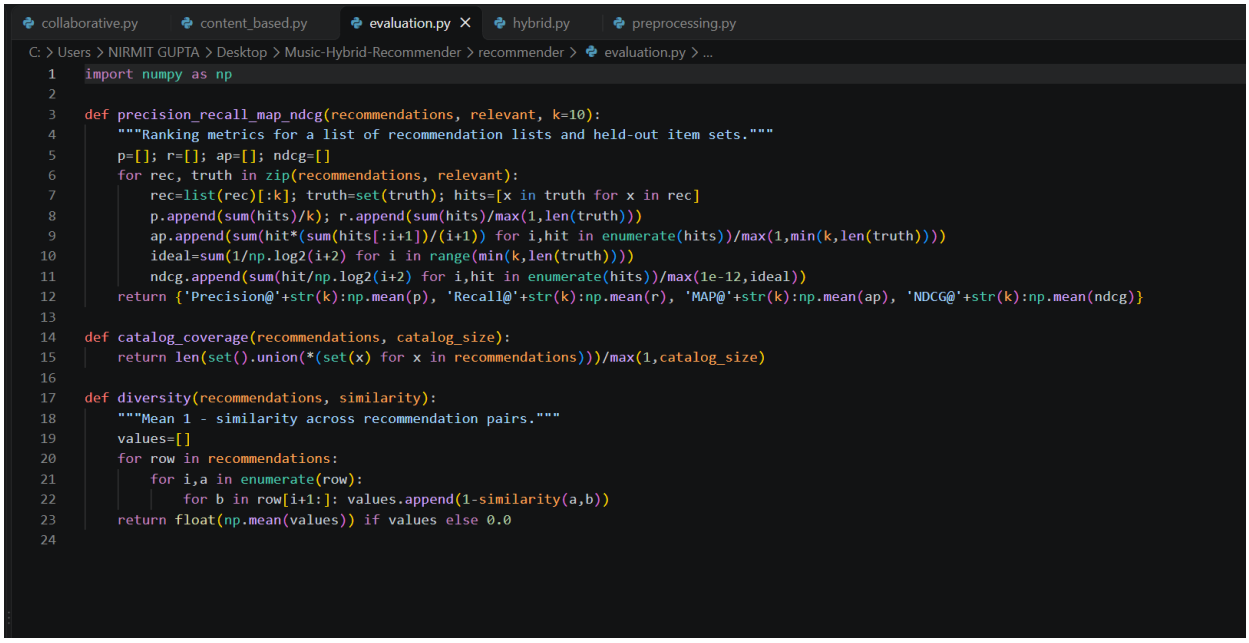
```python
import numpy as np
import pandas as pd

def minmax(values):
    values = pd.Series(values, dtype=float).fillna(0)
    span = values.max() - values.min()
    return (values-values.min())/span if span > 0 else pd.Series(0.0, index=values.index)

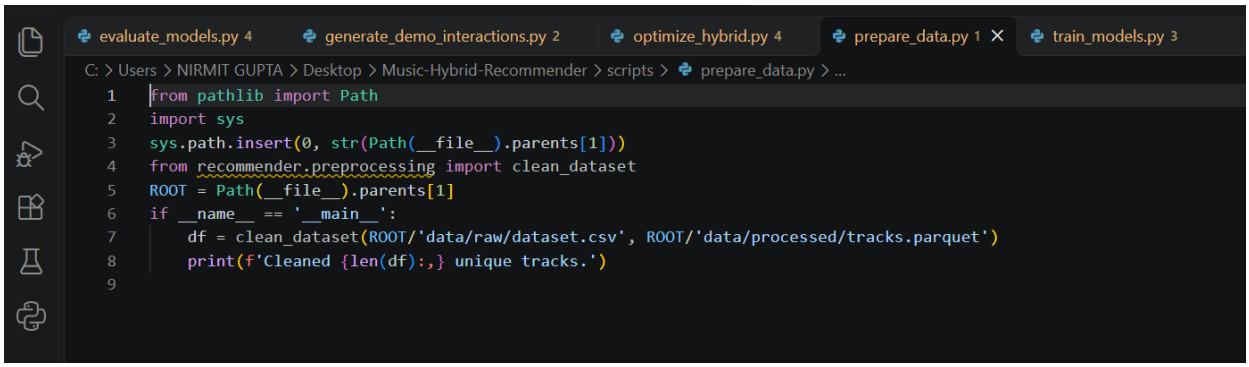
class HybridRecommender:
    def __init__(self, data, content, collaborative): self.data, self.content, self.collaborative = data.reset_index(drop=True), content, collaborative
    def _rank(self, content_scores, collab_scores, candidates, top_n, alpha=.5, genre=None, seed=None):
        candidates = pd.Index(candidates).unique()
        if genre is not None: candidates = candidates[self.data.iloc[candidates].track_genre.to_numpy() == genre]
        if seed is not None: candidates = candidates[candidates != self.content.id_to_index[seed]]
        c=content_scores.reindex(candidates).fillna(0)
        k=collab_scores.reindex(candidates).fillna(0)
        out = self.data[self.data["track_id"].isin(candidates)][["track_id", "track_name", "artists", "track_genre", "popularity"]].copy()
        out["content_score"] = minmax(c).to_numpy(); out["collaborative_score"] = minmax(k).to_numpy()
        out["hybrid_score"] = alpha*out.content_score + (1-alpha)*out.collaborative_score
        return out.sort_values(["hybrid_score","popularity"], ascending=False).drop_duplicates("track_id").head(top_n).reset_index(drop=True)
    def recommend_by_song(self, track_id, top_n=10, alpha=.6):
        if track_id not in self.content.id_to_index: raise KeyError(f"Unknown track_id: {track_id}")
        cs_initial = self.content.scores_for_track(track_id)
        ks_initial = self.collaborative.scores_for_track(track_id, top_k=200)
        if ks_initial.empty or ks_initial.sum() == 0:
            return self._rank(cs_initial, pd.Series(0.0, index=cs_initial.index), cs_initial.index, top_n, alpha=1.0, seed=track_id)
        candidates = cs_initial.index.union(ks_initial.index)
        candidate_matrix_indices = [self.content.id_to_index[tid] for tid in candidates if tid in self.content.id_to_index]
        cs_full = self.content.scores_for_track(track_id, candidates=candidate_matrix_indices)
        ks_full = ks_initial.reindex(cs_full.index).fillna(0)
        return self._rank(cs_full, ks_full, cs_full.index, top_n, alpha, seed=track_id)
    def recommend_by_genre(self, genre, top_n=10, alpha=.5):
        if genre not in set(self.data.track_genre): raise KeyError(f"Unknown genre: {genre}")
        candidates = self.data.index[self.data.track_genre == genre]
        # Transparent content score: genre constraint plus scaled popularity.
        cs = pd.Series(self.data.iloc[candidates].popularity.to_numpy(), index=candidates)
        ks = self.collaborative.genre_scores(candidates)
        return self._rank(cs, ks, candidates, top_n, alpha, genre=genre)

```

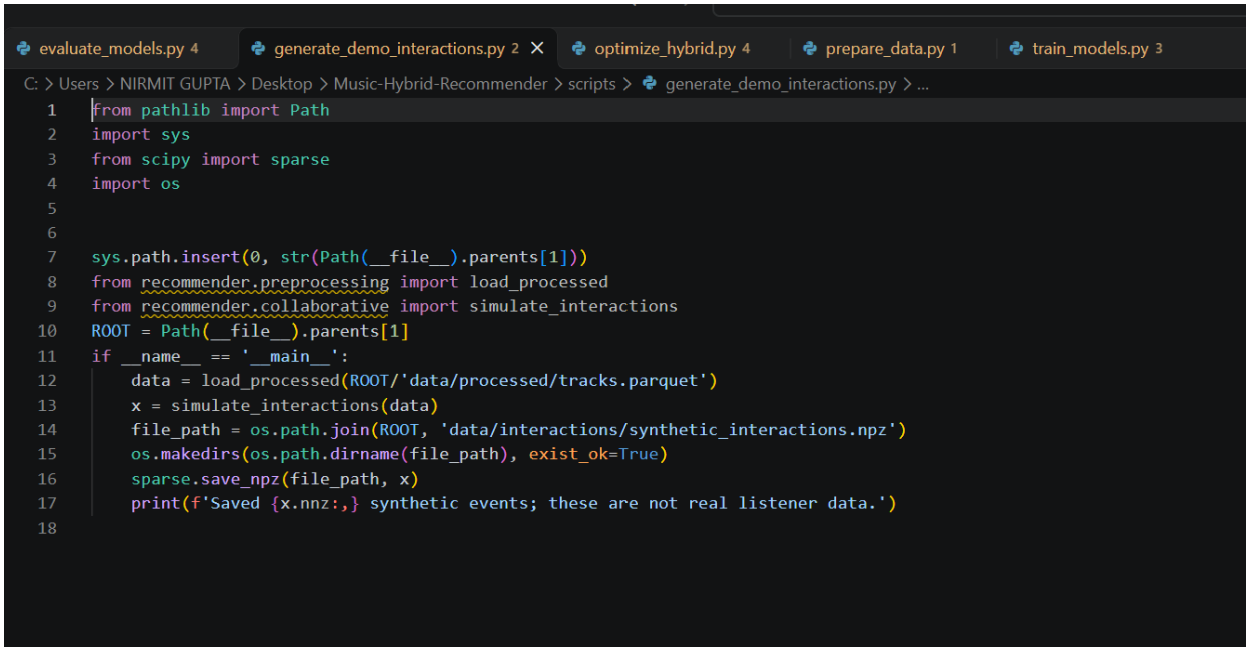
B.5 recommender/evaluation.py



B.6 scripts/prepare_data.py



B.7 scripts/generate_demo_interactions.py

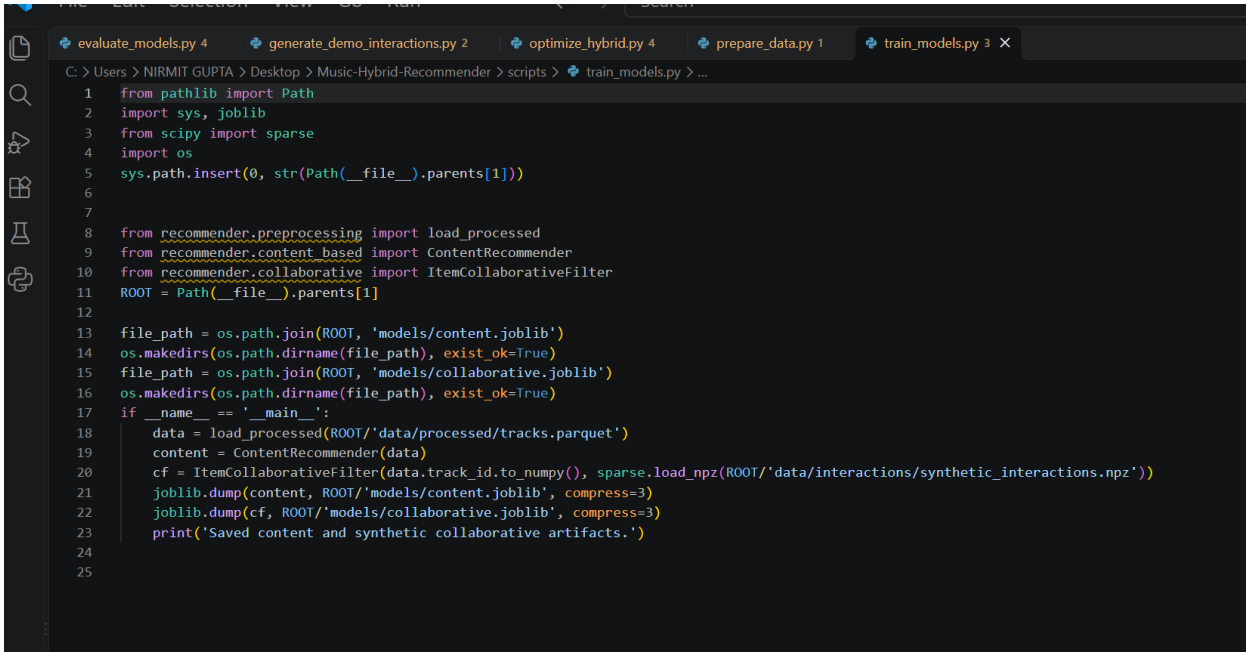


```python
from pathlib import Path
import sys
from scipy import sparse
import os


sys.path.insert(0, str(Path(__file__).parents[1]))
from recommender.preprocessing import load_processed
from recommender.collaborative import simulate_interactions
ROOT = Path(__file__).parents[1]
if __name__ == '__main__':
    data = load_processed(ROOT/'data/processed/tracks.parquet')
    x = simulate_interactions(data)
    file_path = os.path.join(ROOT, 'data/interactions/synthetic_interactions.npz')
    os.makedirs(os.path.dirname(file_path), exist_ok=True)
    sparse.save_npz(file_path, x)
    print(f'Saved {x.nnz:,} synthetic events; these are not real listener data.')

```

B.8 scripts/train_models.py



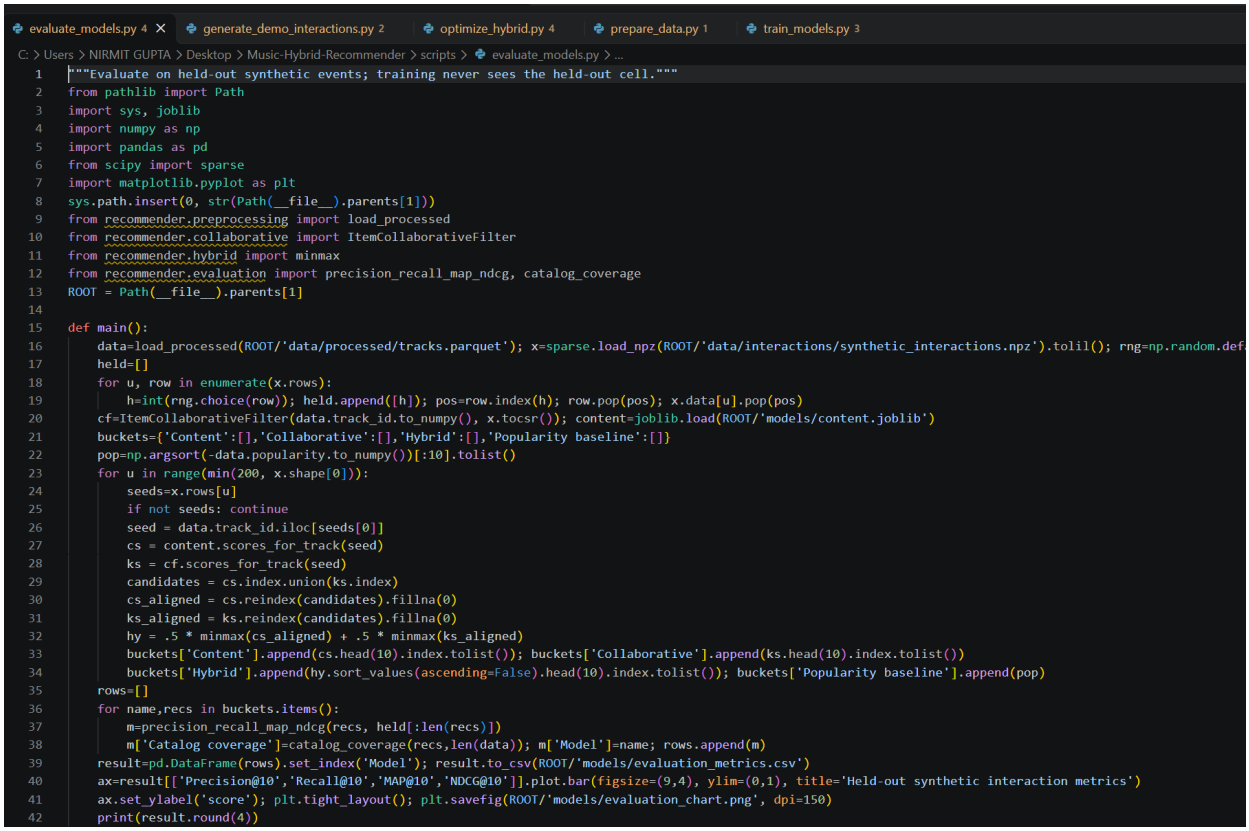
```python
from pathlib import Path
import sys, joblib
from scipy import sparse
import os
sys.path.insert(0, str(Path(__file__).parents[1]))


from recommender.preprocessing import load_processed
from recommender.content_based import ContentRecommender
from recommender.collaborative import ItemCollaborativeFilter
ROOT = Path(__file__).parents[1]

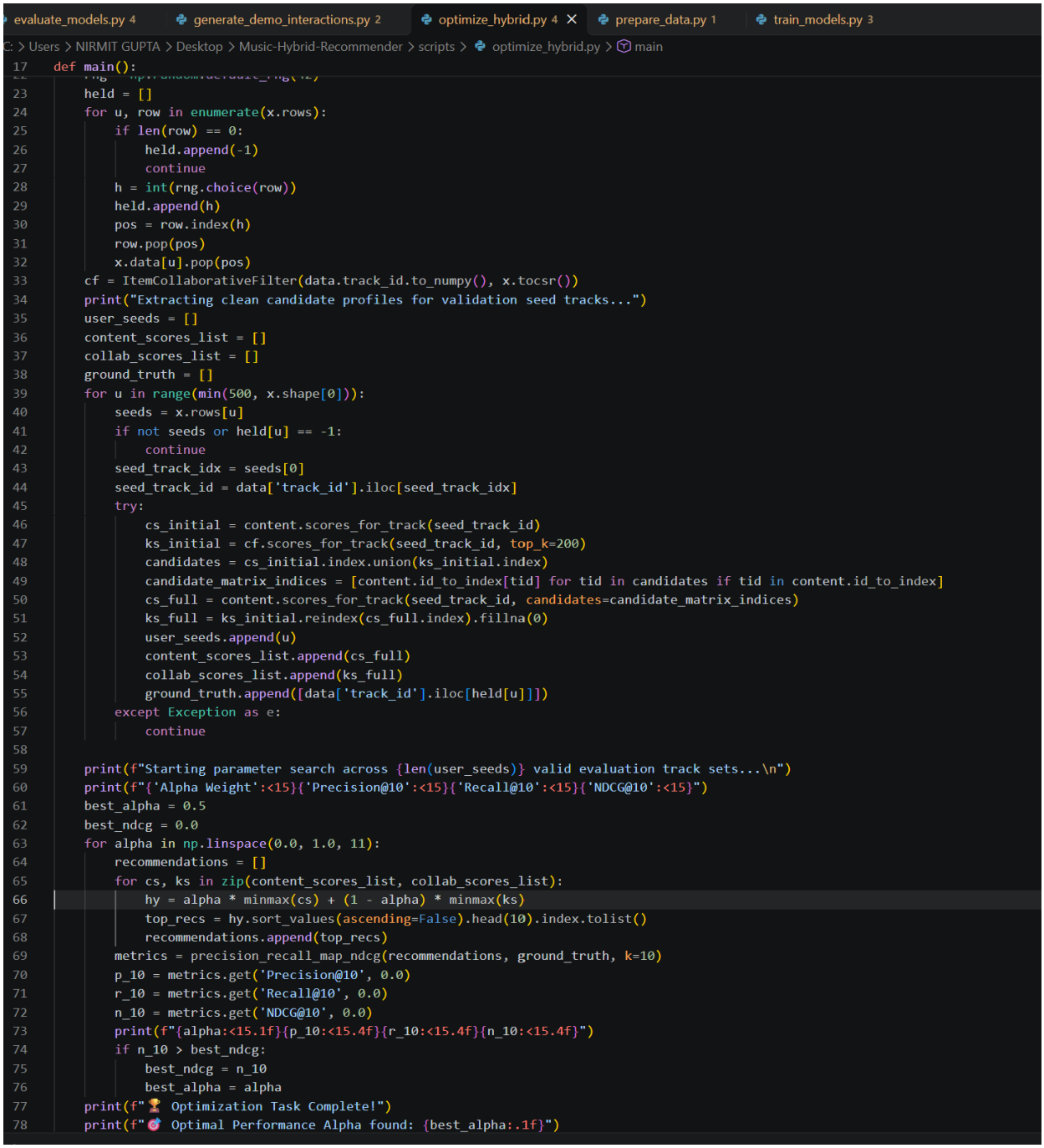
file_path = os.path.join(ROOT, 'models/content.joblib')
os.makedirs(os.path.dirname(file_path), exist_ok=True)
file_path = os.path.join(ROOT, 'models/collaborative.joblib')
os.makedirs(os.path.dirname(file_path), exist_ok=True)
if __name__ == '__main__':
    data = load_processed(ROOT/'data/processed/tracks.parquet')
    content = ContentRecommender(data)
    cf = ItemCollaborativeFilter(data.track_id.to_numpy(), sparse.load_npz(ROOT/'data/interactions/synthetic_interactions.npz'))
    joblib.dump(content, ROOT/'models/content.joblib', compress=3)
    joblib.dump(cf, ROOT/'models/collaborative.joblib', compress=3)
    print('Saved content and synthetic collaborative artifacts.')


```

B.9 scripts/evaluate_models.py



B.10 scripts/optimize_hybrid.py (partial view)



B.11 streamlit_app.py

